

Q1: user结构中, u_cdir 和 u_curdir都是当前工作目录, 有什么不同嘛?

A: u_cdir 是 FCB (文件控制块), u_curdir 是绝对路径名。

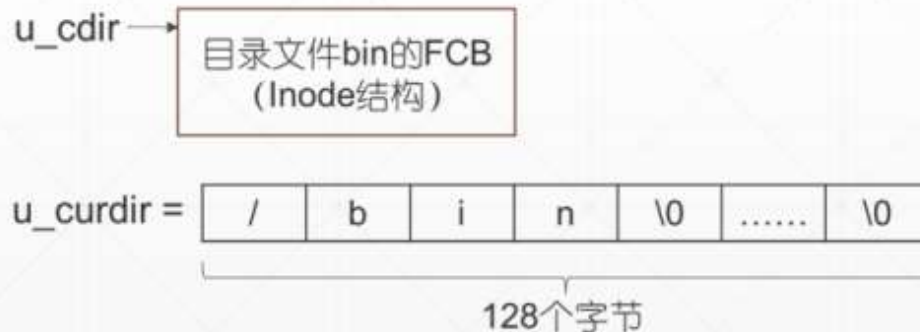
shell进程为例



cd前, shell进程的当前工作目录是根目录 /



cd后, shell进程的当前工作目录是 /bin



备注: Unix系统中一切皆文件。所以, 当前工作目录也是文件。

Unix V6++的FCB是内核数据结构 Inode，登记文件静态属性： 文件长度，分配给它的磁盘扇区号..... 下图红色标出的是最重要的字段。

```

14 class Inode
15 {
130     /* Members */
131 public:
    unsigned int i_flag; /* 状态的标志位，定义见enum INodeFlag */
    unsigned int i_mode; /* 文件工作方式信息 */
    int i_count; /* 引用计数 */
    int i_nlink; /* 文件联结计数，即该文件在目录树中不同路径名的数量 */
    short i_dev; /* 外存inode所在存储设备的设备号 */
    int i_number; /* 外存inode区中的编号 */
    short i_uid; /* 文件所有者的用户标识数 */
    short i_gid; /* 文件所有者的组标识数 */
    int i_size; /* 文件大小，字节为单位 */
    int i_addr[10]; /* 用于文件逻辑块好和物理块好转换的基本索引表 */
    int i_lastr; /* 存放最近一次读取文件的逻辑块号，用于判断是否需要预读 */
148 };
    
```

RWXRWX 在 i_mode 里，是 0~8bit

文件主 i_uid

文件长度

分配给文件的磁盘扇区号

Q2: 打开文件结构究竟长什么样

- 使用中的每个文件，配一个 Inode 结构。

```
14 class Inode
15 {
16     /* Members */
17 public:
18     unsigned int i_flag; /* 状态的标志位, 定义见enum INodeFlag */
19     unsigned int i_mode; /* 文件工作方式信息 */
20
21     int i_count; /* 引用计数 */
22     int i_nlink; /* 文件联结计数, 即该文件在目录树中不同路径名的数量 */
23
24     short i_dev; /* 外存inode所在存储设备的设备号 */
25     int i_number; /* 外存inode区中的编号 */
26
27     short i_uid; /* 文件所有者的用户标识数 */
28     short i_gid; /* 文件所有者的组标识数 */
29
30     int i_size; /* 文件大小, 字节为单位 */
31     int i_addr[10]; /* 用于文件逻辑块好和物理块好转换的基本索引表 */
32
33     int i_lastr; /* 存放最近一次读取文件的逻辑块号, 用于判断是否需要预读 */
34 };
```


文件会话

- 文件会话 是对文件系统的一次完整访问过程。始于open，终于close。例：

备注，open开启一次文件会话

```
int main()
{
    int fd1, fd2;

    fd1 = Open("foo.txt", O_RDONLY, 0); } 第1个文件会话
    Close(fd1);
    fd2 = Open("baz.txt", O_RDONLY, 0); } 第2个文件会话
    printf("fd2 = %d\n", fd2);
    exit(0);
}
```

- 文件会话属性，包括 目标文件，读写请求 和 当前读写位置（文件读写指针）
- File结构，文件会话控制块。一一对应。
 - Open，分配一个File结构
 - Close，释放File结构

```
class File
{
public:
    unsigned int f_flag; // Open系统调用的第2个参数，表示这次文件访问是读还是写
    int         f_count; // File结构的引用计数器
    Inode*      f_inode; // 目标文件的Inode
    int         f_offset; // 文件读写指针，
};
```

creat系统调用也会开启一次会话。f_flag是写

例1, example1进程的打开文件结构

打开文件表u_files

```
int main() // example1.c
```

```
{
```

```
    int fd1, fd2;
```

```
1、 fd1 = Open("foo.txt", O_RDONLY, 0);
```

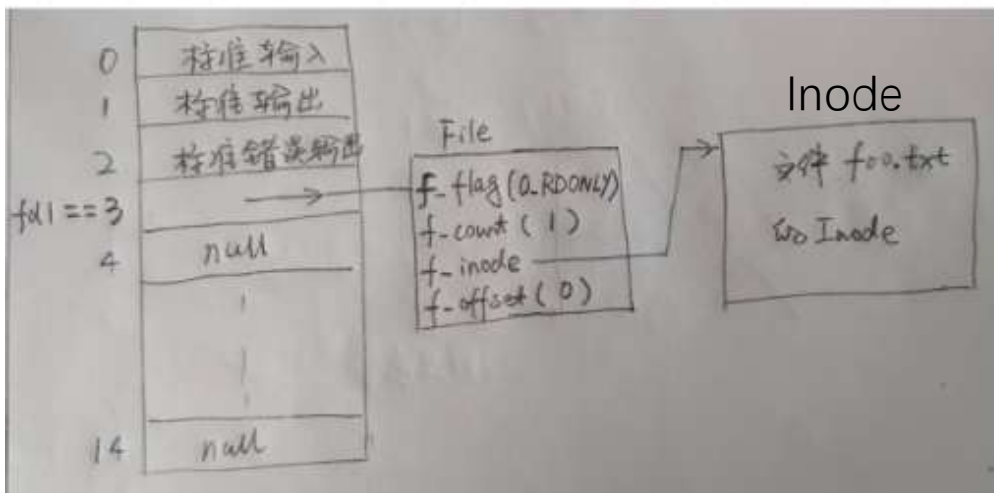
```
2、 Close(fd1);
```

```
3、 fd2 = Open("baz.txt", O_RDONLY, 0);
```

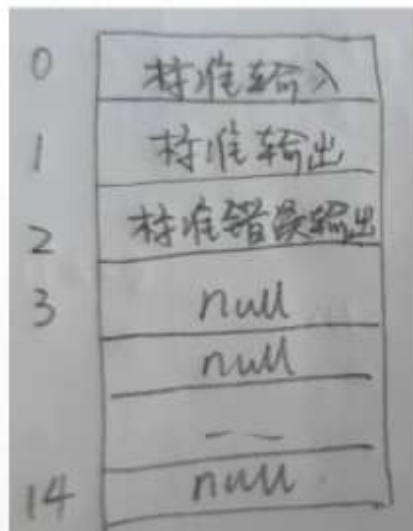
```
    printf("fd2 = %d\n", fd2);
```

```
    exit(0);
```

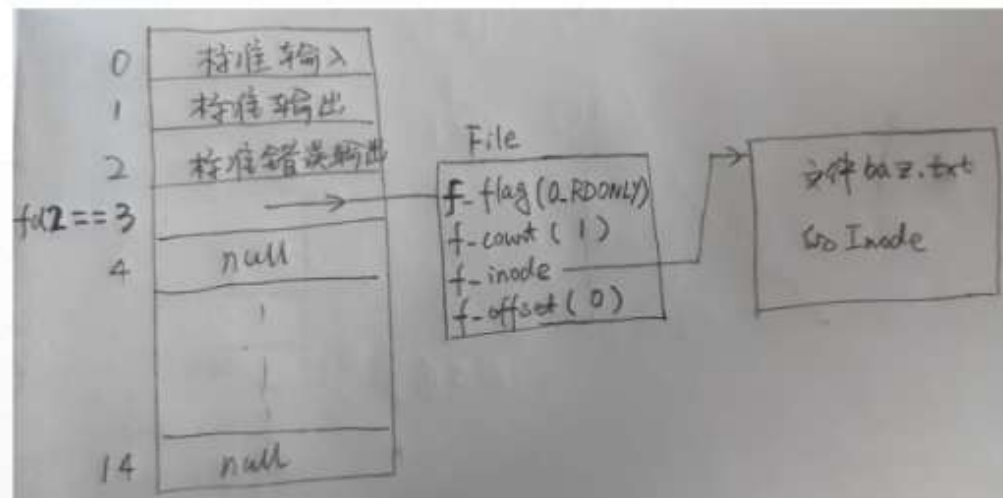
```
}
```



语句 1 执行完毕后



语句 2 执行完毕后



语句 3 执行完毕后